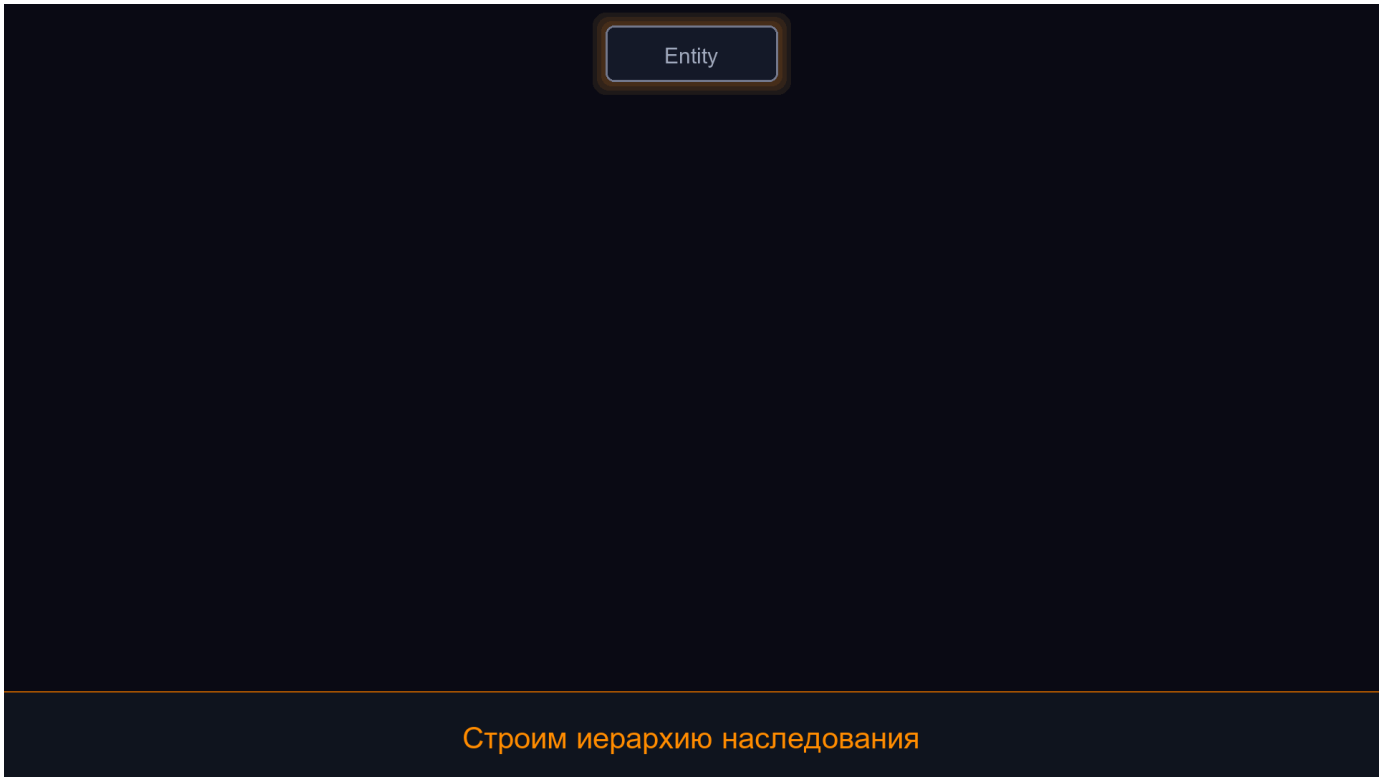
ООП в GameDev

ООП в GameDev



Ок. Мы разрабатываем системы в контексте ООП - они реализуют классы, кто-то от чего-то наследуется, инкапсулируем детали и т.д. и т.п. К примеру:

Python C#

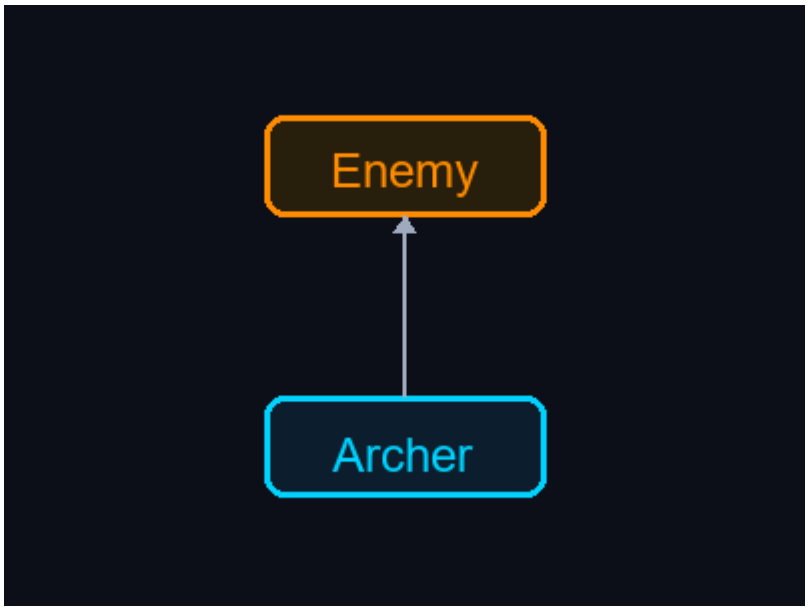
```python
class InventoryItem:
    def __init__(self, name, quantity):
        self.name = name
        self.quantity = quantity

class InventoryManager:
    def __init__(self):
        self.items = {}

    def add_item(self, item):
        if item.name in self.items:
            self.items[item.name].quantity += item.quantity
        else:
```

```
            self.items[item.name] = item

    def remove_item(self, item_name, quantity):
        if item_name in self.items and self.items[item_name].quantity >= qu
            self.items[item_name].quantity -= quantity
            if self.items[item_name].quantity == 0:
                del self.items[item_name]
```

Затем мы из какой-то модели, допустим, MonitorModel обращаемся к данной системе и работаем с ней. Вроде, никаких подвохов.
Однако, стоит ли применять ООП везде? Есть варианты использования и функционального программирования, и Data-Oriented Programming. Но если мы останавливаемся на ООП (а противоречит ли она другим вообще?..), не отвлекаясь на другие парадигмы, то с какими проблемами мы можем столкнуться?

Множественное наследование

Допустим, есть класс Enemy, мы хотим сделать врага-лучника Archer. Для этого создаем класс Archer, ко



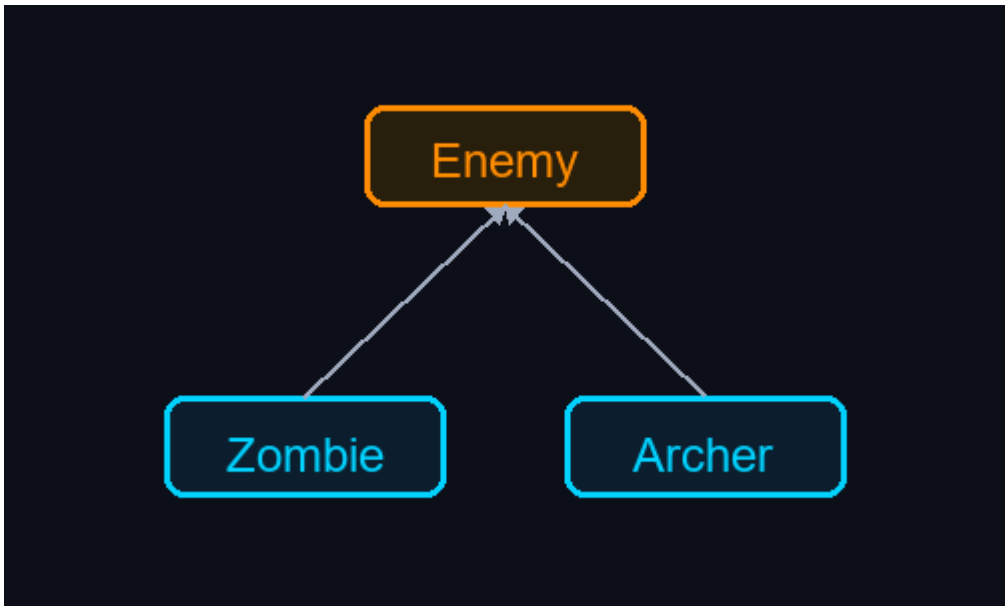
Python C#

```
class Enemy:
    def __init__(self, health, strength):
        self.health = health
        self.strength = strength

class Archer(Enemy):
    def __init__(self, health, strength, attack_range):

```

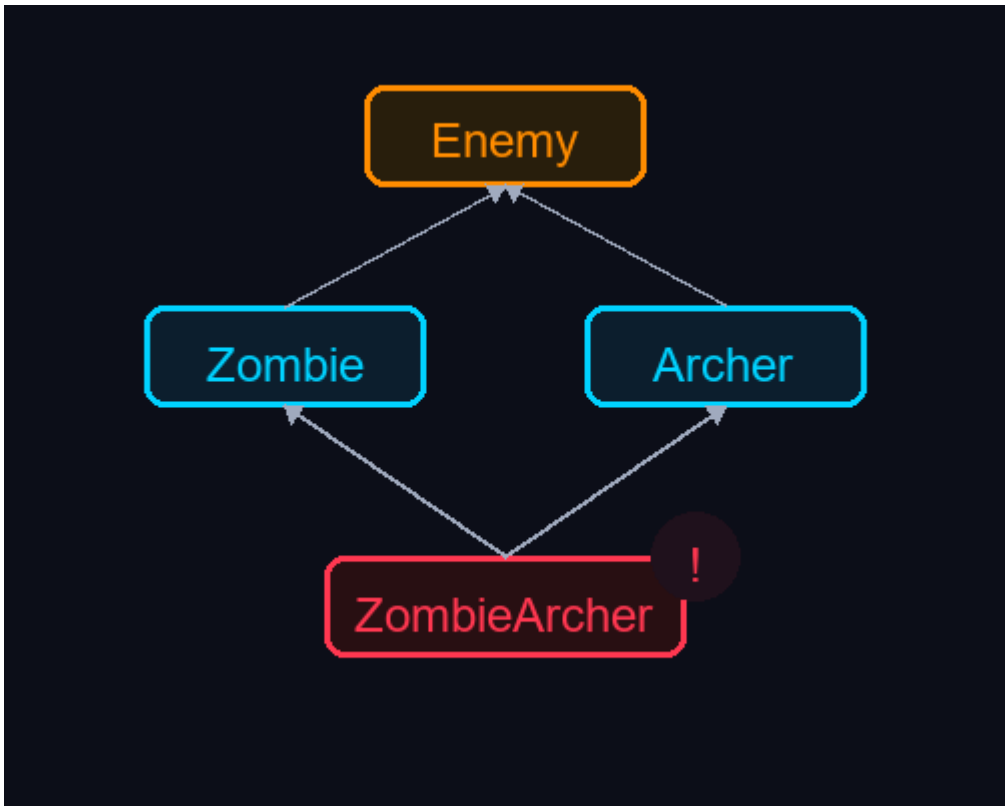
```
        super().__init__(health, strength)
        self.attack_range = attack_range
```

Пока все хорошо.

Идем дальше. Захотели мы создать зомби. Ну, создаем класс Zombie, наследуем его от Enemy.



А потом решили создать зомби-лучника? Тогда нам придется наследоваться от Zombie и от Archer.



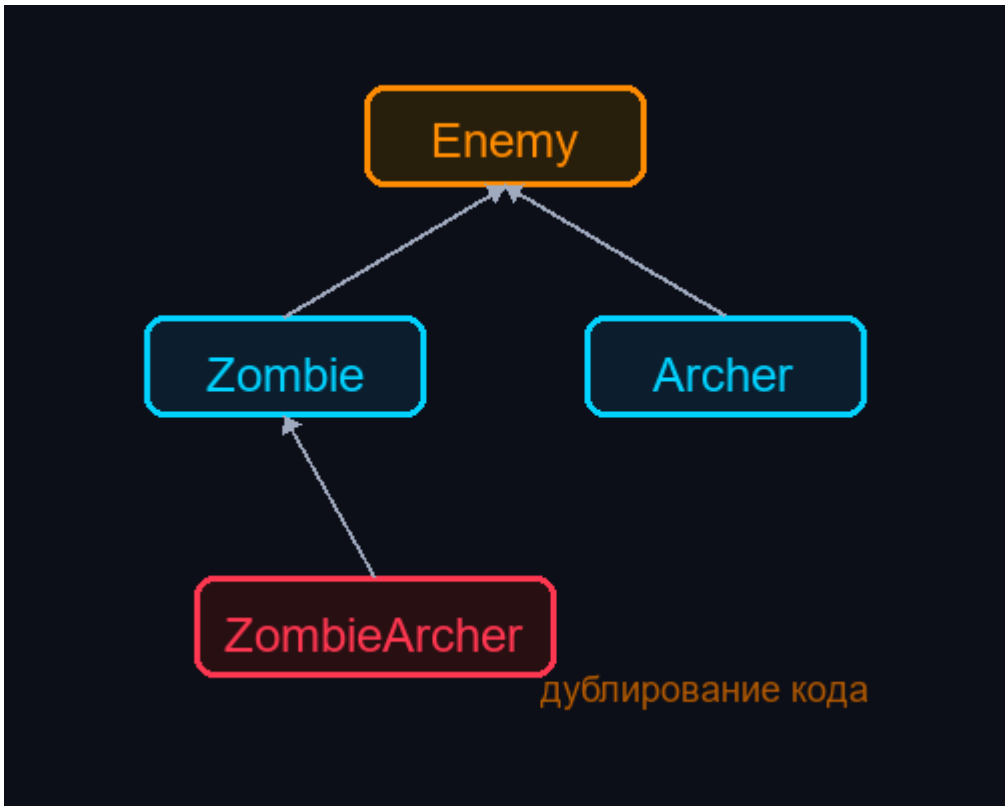
Python C#

В Python множественное наследование допускается, что позволяет классу наследовать функциональность сразу от нескольких родительских классов. Но есть несколько проблем:

- **Diamond Problem**: когда класс D наследуется от двух классов B и C, которые в свою очередь наследуются от одного общего предка A, имеющего метод Test(). И тогда возникает вопрос: при вызове метода Test у экземпляра класса D какая именно реализация метода сработает? Из A, из B, из C? Python дает ответ на этот вопрос (см. MRO), однако с точки зрения архитектуры эта ситуация все еще выглядит не очень удобной для работы (т.к. в реальности у вас будет не 4 класса, а гораздо больше... см. следующую проблему).
- **Сложность поддержки**: множественное наследование усложняет архитектуру классов, делая код более запутанным и трудным для понимания и поддержки, особенно при большом количестве уровней наследования и перекрестных зависимостей.

Поэтому стоит с осторожностью использовать множественное наследование.

Можно, конечно, сделать так, продублировав функционал:



Но это не очень вариант.

Как мы можем исправить эту ситуацию? Использовать интерфейсы в C#? Абстрактные классы и Протоколы в Python? Возможно. Но давайте попробуем сначала отказаться от множественного наследования, а потом посмотреть, как будет выглядеть куча интерфейсов.

Одиночное наследование

Вообще, не каждый объект в игре должен иметь физические свойства. Давайте возьмем в качестве примера дом. Все, что нам от него нужно - это коллизия, т.е. возможность столкновения, а его вес, физические характеристики - нет, поскольку мы не хотим, чтобы он вообще когда-то двигался.

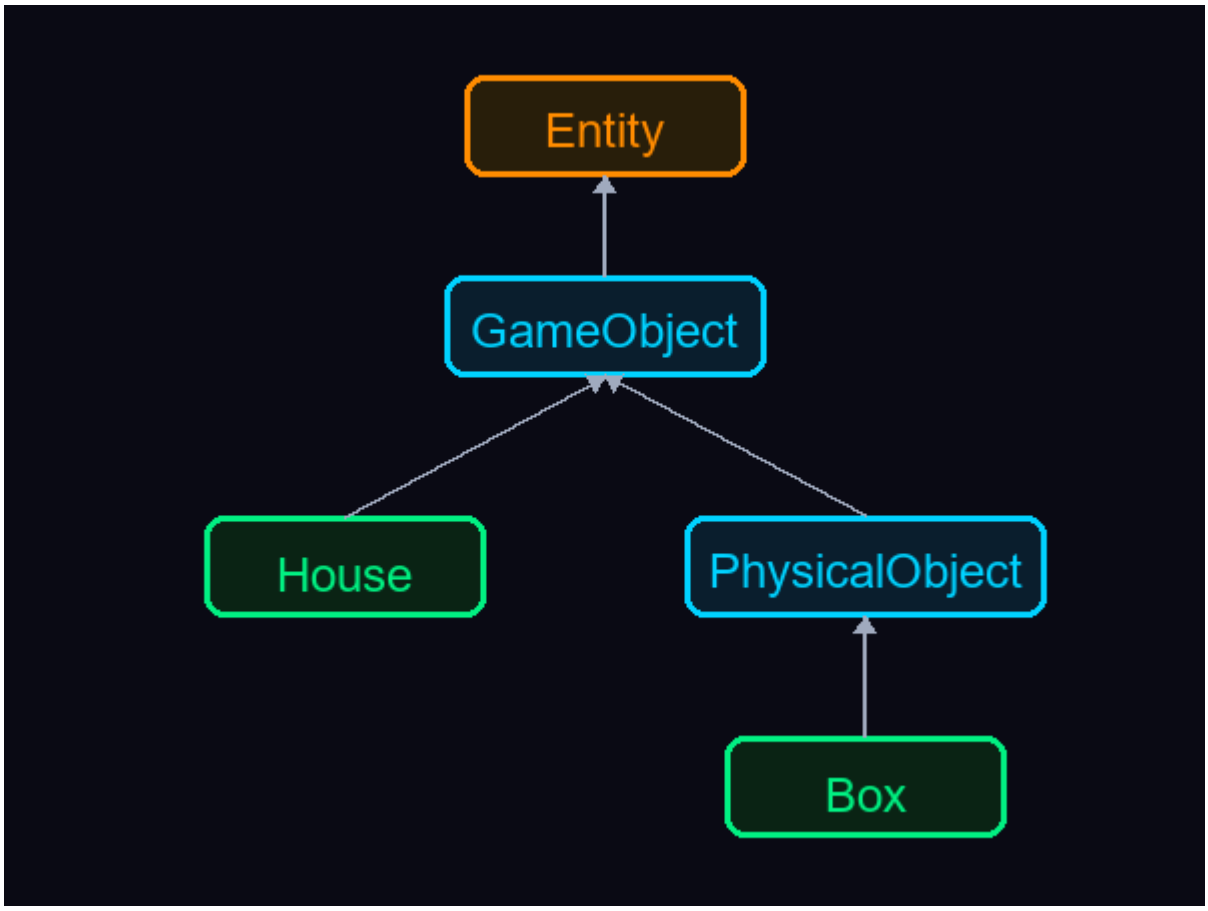
А вот коробочка, которая находится в доме - должна двигаться. Соответственно, у неё должна быть какая-то физическая модель.

Как эта ситуация может лечь на нашу иерархию классов?

В игре есть сущности `Entity`, которые хранят только название. От `Entity` у нас наследуются `GameObjects` - экземпляры которого представляют из себя внутриигровые объекты, которые мы видим в игре. А от `GameObject` наследуются `House` и `Box`.

- **House**: Не требует физическую модель.
- **Box**: Требует физическую модель, так как предполагается его движение.

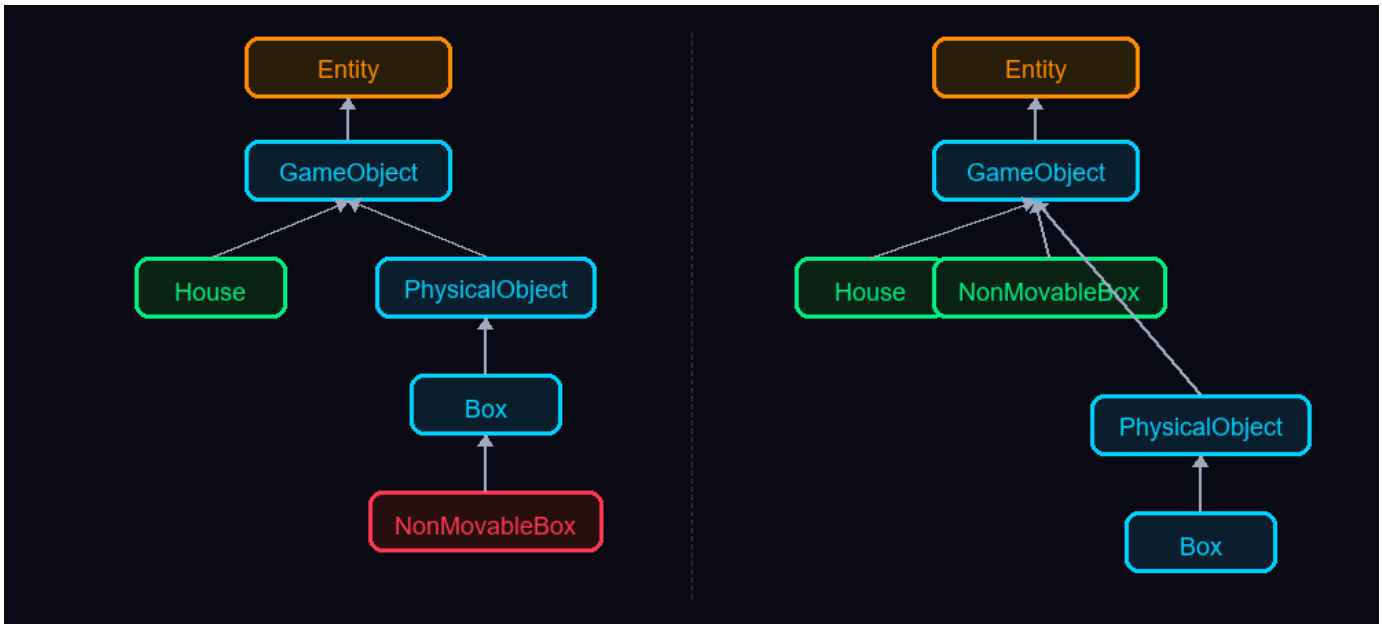
Логично было бы выделить `PhysicalObject`, наследующийся от `GameObject`, а от него наследовать `Box`.



А теперь давайте представим, что нам нужна коробка, которая не будет никак работать с физикой. Для заднего фона (оптимизация своего рода). Получается, нам нужен Box, но который не наследуется от PhysicalObject.

И что тогда делать?

- Вариант `NonMovableBox->GameObject->Entity`: мы дублируем код в двух разных Box.
- Вариант `NonMovableBox -> Box->PhysicalObject->GameObject->Entity`: у нас есть NonMovableBox, который, вроде, является PhysicalObject, но никак не взаимодействует с физикой. Что звучит довольно странно.



Иерархия классов, наследования становится сложнее, запутаннее. Более того, получается так, что вся логика, все особенности поведения Box описаны в нем самом: т.е. это такая большая сущность, которая определяет и коллизию объекта, и его физическую модель, и его координаты и т.д. и т.п. И если мы что-то захотим изменить, тогда нам придется копаться в большом количестве кода. А поскольку он написан в одном классе, велик соблазн залезть из физической части в коллизию и наоборот, запутав и усложнив код.

И чем сложнее механики в игре, тем сложнее будет иерархия наследования. В какой-то момент вы будете наследоваться от 10 и более классов, или реализовывать 10 и более интерфейсов, некоторые из которых могут быть объединением других. Легко запутаться и сложно сказать, как изменения того или иного класса или интерфейса повлияют на другие сущности в вашей игре.

ООП и MVC

Более того, если мы применяем ООП и MVC, то отображаемые объекты будут разбиты на три независимые части, каждая из которых будет реализовывать разные интерфейсы и наследоваться от разных классов. Что... тоже не звучит как упрощение.

- Это не означает, что нельзя применять MVC (или другие паттерны из этого семейства).
- Это не означает, что нельзя применять ООП, или нельзя применять их вместе.

Выбирая архитектуру, вы должны иметь в виду плюсы и минусы тех или иных подходов.

Так, MVC рассматривает вашу игру как большую систему, и говорит, что эта система делится на три части. Как именно эти части реализуются - отдельный вопрос. И на этот вопрос отвечает ООП: когда нам нужна какая-то сущность, мы создаем класс и т.д.

Т.е. MVC и ООП не противопоставлены друг другу, это не альтернативы и не синонимы, они работают вместе.

Но что сказать по поводу наследования в ООП и его проблемы при использовании в игре?

Во-первых, не факт, что вы столкнетесь с ними.

Во-вторых, если же вы планируете развивать вашу игру, вы знаете, что она будет расти, т.е. кол-во разнообразных сущностей тоже будет расти, их поведение и отношения будут сложными, вы знаете, что хотите сделать более гибкую систему именно с запасом на будущее, то можно рассмотреть ECS - Entity Component System, которая исходит из идеи

"Композиция вместо наследования".

Что	Где подробнее
ECS - композиция вместо наследования	ECS и EC
Архитектурные паттерны (MVC, MVP, MVVM)	Архитектура, MVC
Типичные ошибки ООП в играх	Типичные ошибки
Конечные автоматы для управления состояниями	Конечные автоматы

Powered by Wiki.js